Hashing

- **Tables**
- **Direct address tables**
- **Hash tables**
- **Collision and collision resolution**
- **Chaining**

Introduction

- Many applications require a dynamic set that supports dictionary operations.
- Example: a compiler maintaining a symbol table where keys correspond to identifiers
- Hash table is a good data structure for implementing dictionary operations
- Although searching can take as long as a linked list implementation i.e. O(n) in worst case.

Introduction

- With reasonable assumptions it can take O(1) time.
- In practice hashing performs extremely well.
- A hash table is a generalization of an ordinary array where direct addressing takes O(1) time.
- When the actual keys is NOT small relative to the total number of keys, hashing is an effective alternative.
- A key can be accessed using an array index, or is computed.

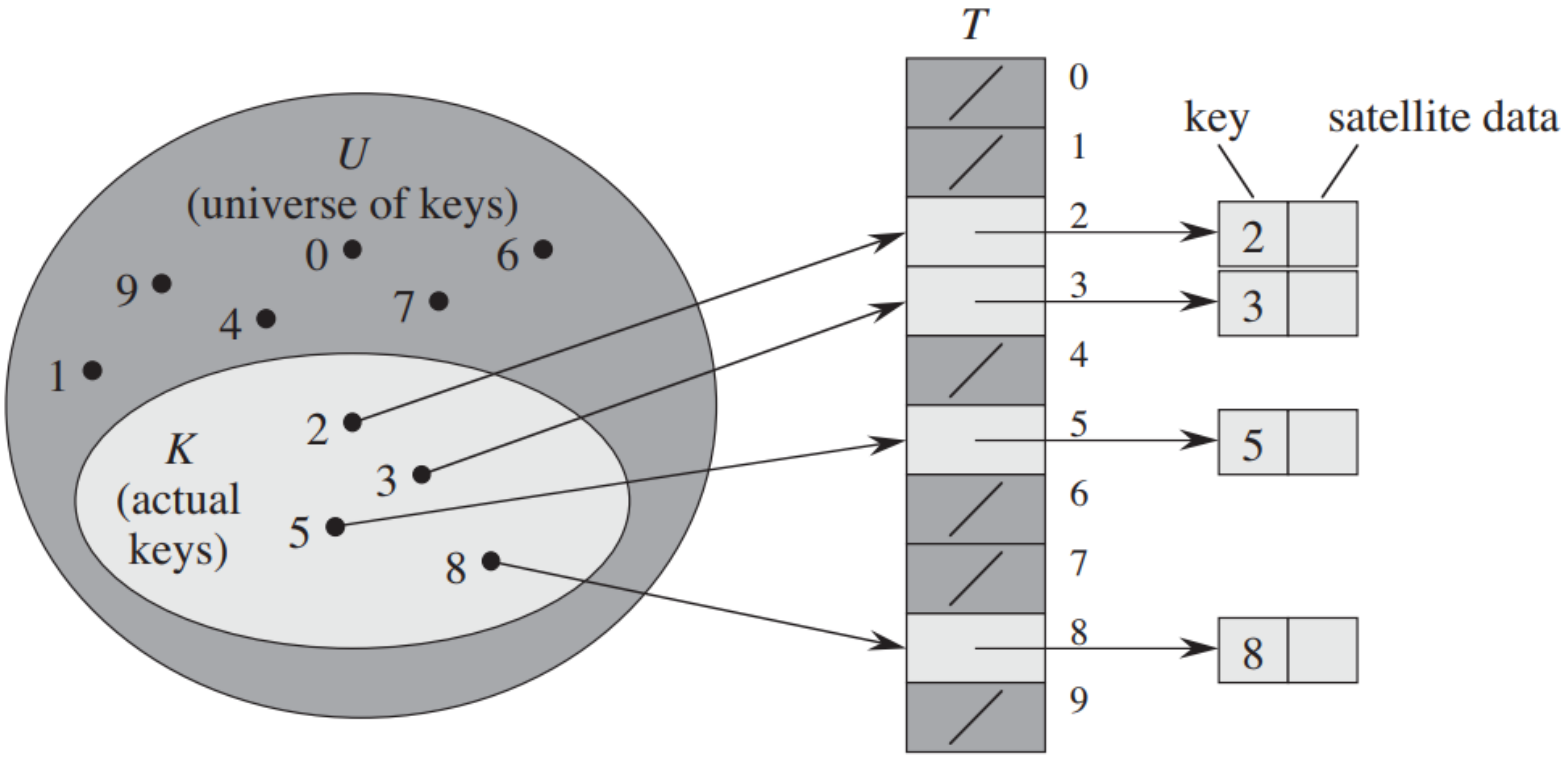
What are Tables?

- Table is an abstract storage device that contains table entries
- Each table entry contains a unique key *k*.
- Each table entry may also contain some information, *I*, associated with its key.
- A table entry is an ordered pair *(K, I)*

Direct Addressing

- Suppose:
  - The range of keys is 0..m-1
  - Keys are distinct
- The idea:
  - Set up an array T[0..m-1] in which
    - T[i] = x if $x \in T$ and key[x] = i
    - T[i] = NULL otherwise
  - This is called a *direct-address table*
    - Operations take O(1) time!

Direct Addressing



Advantages with Direct Addressing

- Direct Addressing is the most efficient way to access the data since.
- It takes only single step for any operation on direct address table.
- It works well when the Universe U of keys is reasonable small.

Difficulty with Direct Addressing

When the universe U is very large…

- Storing a table T of size U may be impractical, given the memory available on a typical computer.
- The set K of the keys actually stored may be so small relative to U that most of the space allocated for T would be wasted.

An Example

- A table, 50 students in a class.
- The key, 9-digit *SSN*, used to identify each student.
- Number of different 9-digit number=10^9
- The fraction of actual keys needed. $50/10^9$, 0.000005%
- Percent of the memory allocated for table wasted, 99.999995%

An ideal table needed!

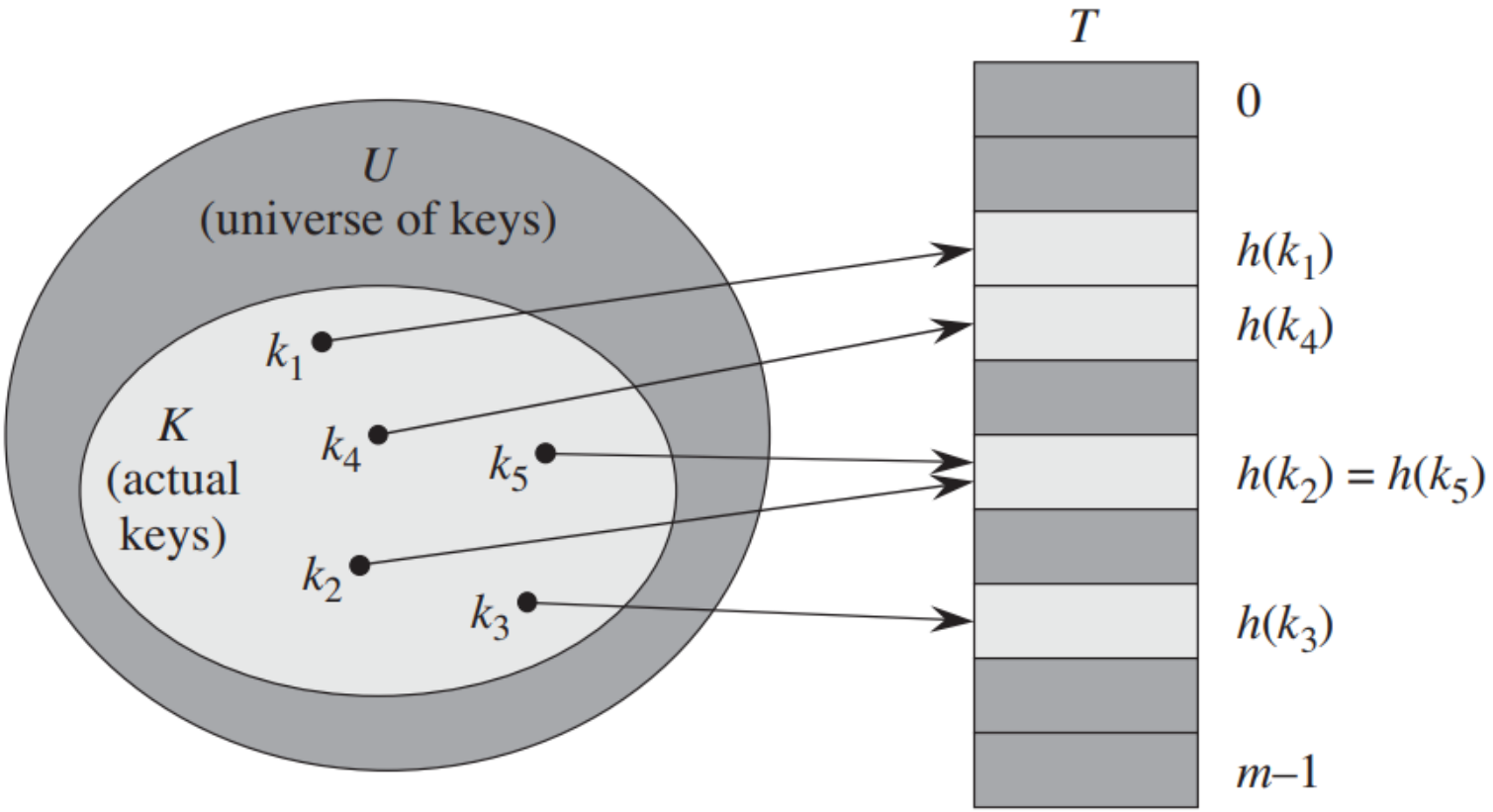
- The table should be of small fixed size.

- Any key in the universe should be able to be mapped in the slot into table, using some mapping function

Hash Tables

- Definition: the ideal table data structure is merely an array of some fixed size, containing the elements.
- Consist : an array and a mapping function (known as hash function)
- Used for performing insertion, deletion and lookup on average in constant time.

Hash Tables


T
U
(universe of keys)
K
(actual
keys)
k_1
k_4
k_5
k_2
k_3
0
$h(k_1)$
$h(k_4)$
$h(k_2) = h(k_5)$
$h(k_3)$
$m-1$

Compared to direct addressing

- **Advantage: Requires less storage and runs in *O(1)* time.**
- **Comparison**

	Storage Space	Storing k
Direct Addressing	**|U|**	**Store in slot k**
Hashing	**m**	**Store in slot h(k)**

Collision

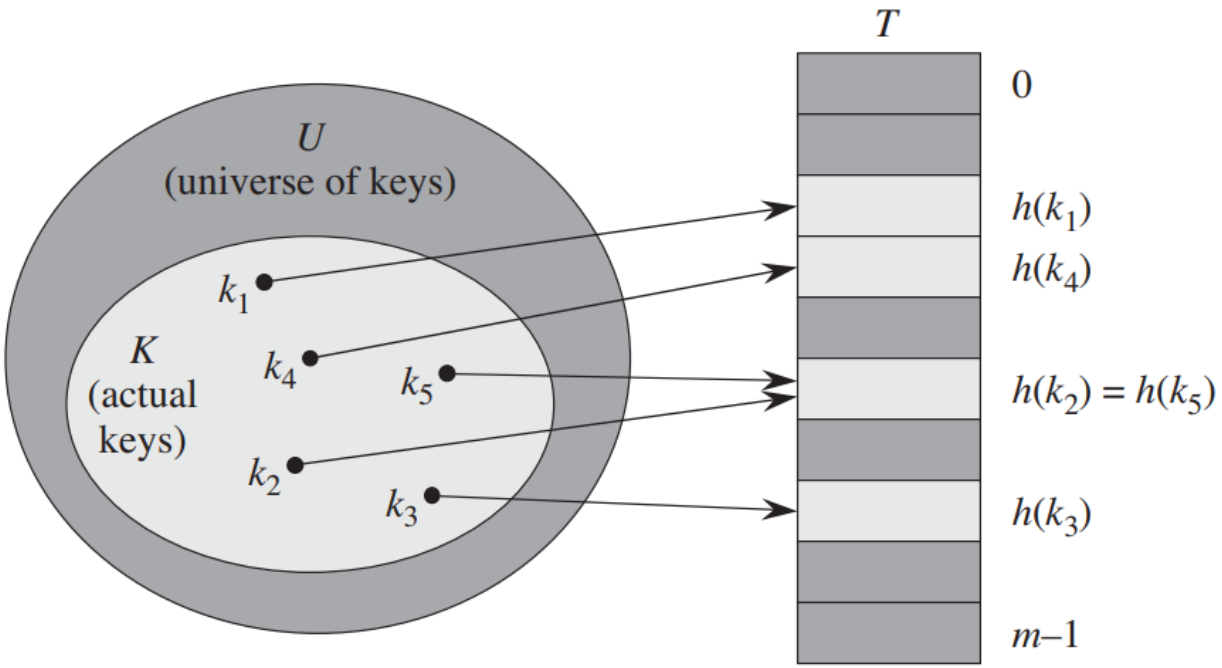


Figure 11.2 Using a hash function h to map keys to hash-table slots. Because keys k_2 and k_5 map to the same slot, they collide.

Resolving Collisions

- *How can we solve the problem of collisions?*

- Solution 1: *Chaining*

- Solution 2: *Open addressing*

Chaining!

- Put all the elements that hash to same slot in a linked list.
- Worst case : All *n* keys hash to the same slot resulting in a linked list of length *n*, running time: *O(n)*
- Best and Average time: *O(1)*

Collision by Chaining

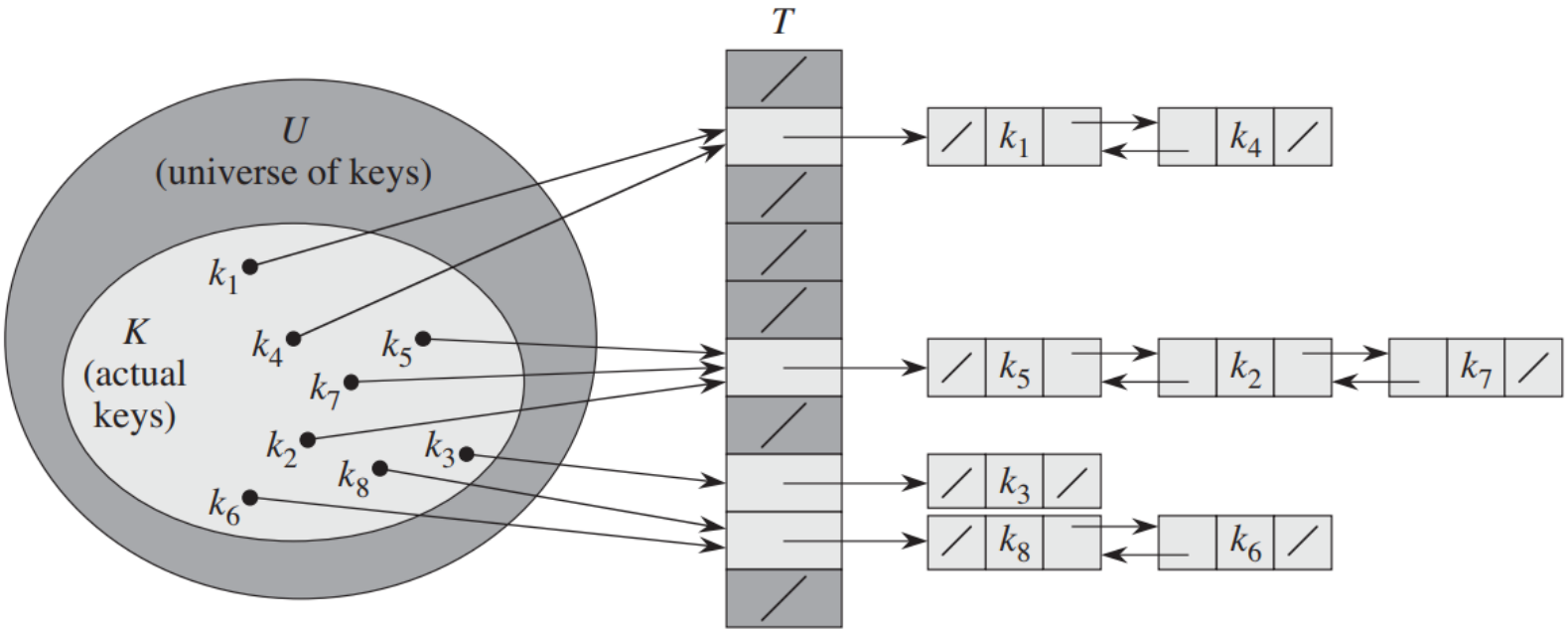
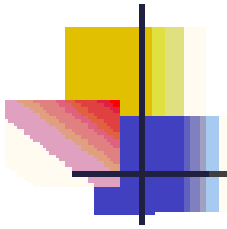
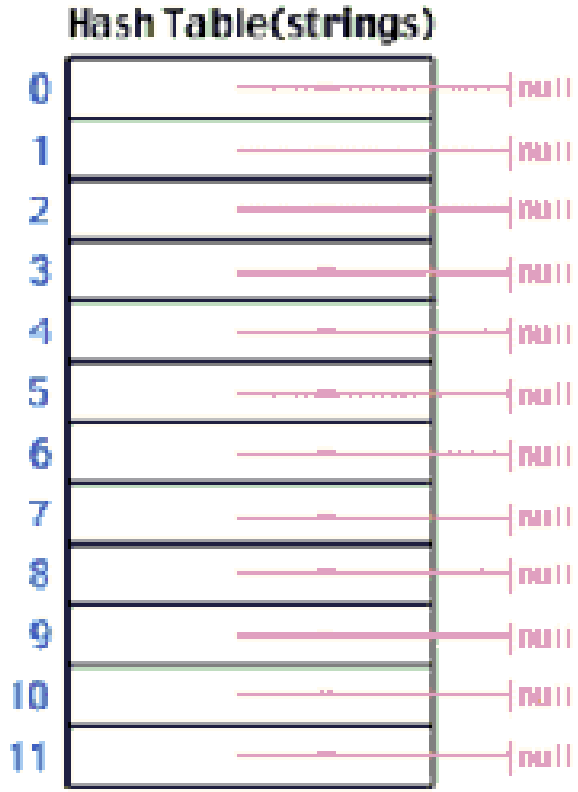


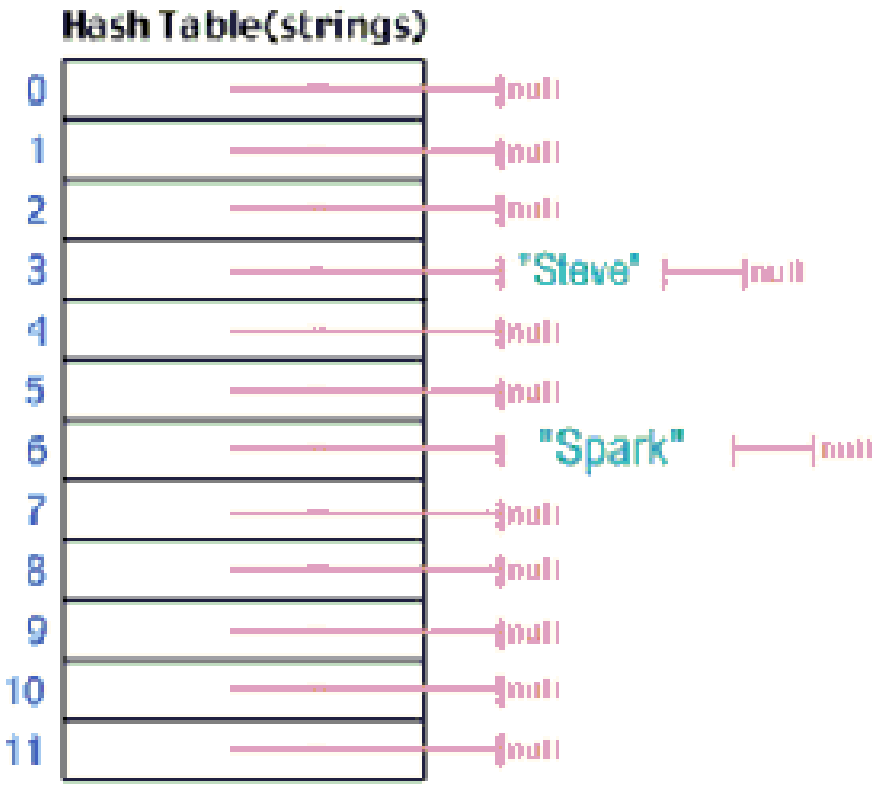
Figure 11.3 Collision resolution by chaining. Each hash-table slot $T[j]$ contains a linked list of all the keys whose hash value is j. For example, $h(k_1) = h(k_4)$ and $h(k_5) = h(k_7) = h(k_2)$. The linked list can be either singly or doubly linked; we show it as doubly linked because deletion is faster that way.



Example



Cont'



Analysis of Chaining

- Assume *simple uniform hashing*: each key in table is equally likely to be hashed to any slot
- Given n keys and m slots in the table: the *load factor* $\alpha = n/m$ = average # keys per slot
- *What will be the average cost of an unsuccessful search for a key?*

  $O(1+\alpha)$

What will be the average cost of a successful search?

A: $O(1 + \alpha/2) = O(1 + \alpha)$

Analysis of Chaining Continued

- So the cost of searching = $O(1 + \alpha)$
  - *If the number of keys n is proportional to the number of slots in the table, what is α?*
    - $\alpha = O(1)$
  - In other words, we can make the expected cost of searching constant if we make α constant

Hash Tables

- Nature of keys
- Hash functions
- Division method
- Multiplication method
- Open Addressing (Linear and Quadratic probing, Double hashing)

Nature of Keys

- Most hash functions assume that universe of keys is the set N = {0, 1, 2,…} of natural numbers
- If keys are not N, ways to be found to interpret them as N
- A character key can be interpreted as an integer expressed in suitable Radix notation.

Nature of Keys

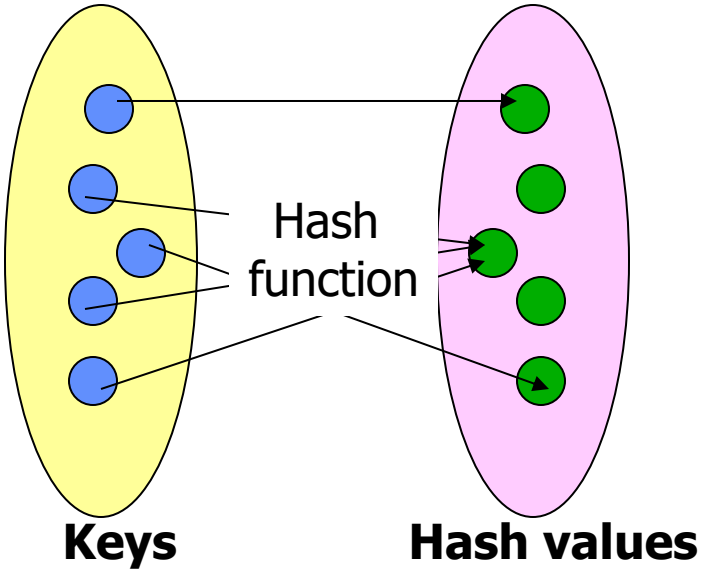
- Example: The identifier pt might be interpreted as a pair of decimal integers (112, 116) as p = 112 and t = 116 in ASCII notation. What is the problem?

- Using a product/addition of ASCII codes is indifferent to the order of characters

- Solution: Using 128-radix notation this becomes (112.128) + 116 = 14,452

What is a Hash function?

A hash function is a mapping between a set of input values (Keys) and a set of integers, known as hash values.



The properties of a good hash function

- Rule1: The hash value is fully determined by the data being hashed.
- Rule2: The hash function uses all the input data.
- Rule3: The hash function uniformly distributes the data across the entire set of possible hash values.
- Rule4: The hash function generates very different hash values for similar strings.

An example of a hash function

```
int hash(char *str, int table_size)
{
    int sum=0;
  //sum up all the characters in the string
   for(;*str; str++) sum+=*str
  //return sum mod table_size
 return sum%table_size;
}
```

Analysis of example

- Rule1: Satisfies, the hash value is fully determined by the data being hashed, the hash value is just the sum of all input characters.

- Rule2: Satisfies, Every character is summed.

Analysis of example (contd.)

- Rule3: Breaks, from looking at it, it is not obvious that it doesn't uniformly distribute the strings, but if you were to analyze this function for larger input string, you will see certain statistical properties which are bad for a hash function.

- Rule4: Breaks, hash the string "CAT", now hash the string "ACT", they are the same, a slight variation in the string should result in different hash values, but with this function often they don't.

Methods to create hash functions

- Division method
- Multiplication method

Division method

The division method requires two steps.

1. The key must be transformed into an integer.

2. The value must be telescoped into range 0 to m-1

Division method…

- We map a key *k* into one of the *m* slots by taking the remainder of *k* divided by m, so the hash function is of form

$$h(k) = k \bmod m$$

- For example , if *m=12*, key is *100* then *h(k)=100 mod 12= 4*.

- Advantage?

Restrictions on value of m

M should not be a power of 2, since if $m=2^p$ then h(k) is just the p lowest order bits of k.

Disadvantage!

Key	Binary	K mod 8
8	1000	0
7	111	7
12	1100	4
34	100010	2
56	111000	0
78	1001110	6
90	1011010	2
23	10111	7
45	101101	5
67	1000011	3

Restrictions on value of **m**

- Unless it is known that probability distribution on keys makes all lower order p-bit patterns equally likely,

- It is better to make the hash function dependent on all the bits of the key.

Good value of **m**

- Power of 10 should be avoided, if application deals with decimal numbers as keys.
- Good values of *m* are primes not close to the exact powers of 2 (or 10).

Multiplication method

- Using a random real number *f* in the range *[0,1)*.
- The fractional part of the product *f*key* yields a number in the range *0 to 1*.
- When this number is multiplied by m (hash table size), the integer portion of the product gives the hash value in the range *0 to m-1*

More on multiplication method

- Choose $m = 2^P$
- For a constant A, $0 < A < 1$:
- $h(k) = \lfloor m\,(kA - \lfloor kA \rfloor) \rfloor$
- Value of A should not be close to 0 or 1
- *Knuth* says good value of A is 0.618033
- If $k=123456$, $m=10000$, *and* A as above

  $h(k) = \lfloor 10000.(123456*A - \lfloor 123456*A \rfloor)$

  $= \lfloor 10000.0.0041151 \rfloor$

  $=41$

Hashing with Open Addressing

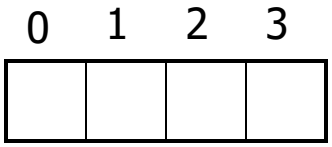
- So far we have studied hashing with chaining, using a linked-list to store keys that hash to the same location.
- Maintaining linked lists involves using pointers which is complex and inefficient in both storage and time requirements.
- Another option is to store all the keys directly in the table. This is known as open addressing, where collisions are resolved by systematically examining other table indexes, i_0, i_1, i_2, … until an empty slot is located.

Open addressing

- Another approach for collision resolution.
- All elements are stored in the hash table itself (so no pointers involved as in chaining).
- To insert: if slot is full, try another slot, and another, until an open slot is found (*probing*)
- To search, follow same sequence of probes as would be used when inserting the element

Open Addressing

- The key is first mapped to a slot:
  $\text{index} = i_0 = h_1(k)$
- If there is a collision subsequent probes are performed:
  $$i_{j+1} = (i_j + c) \bmod m \quad \text{for} \quad j \geq 0$$
- If the offset constant, *c* and *m* are not relatively prime, we will not examine all the cells. Ex.:
  - Consider m=4 and c=2, then only every other slot is checked. When *c*=1 the collision resolution is done as a linear search. This is known as linear probing.



Insertion in hash table

```
HASH_INSERT(T,k)
1   i ← 0
2   repeat j ← h(k,i)
3         if T[j] = NIL
4               then T[j] = k
5                    return j
6         else i ←  i +1
7   until i = m
8   error " hash table overflow"
```

Searching from Hash table

```
HASH_SEARCH(T,k)
1       i ← 0
2   repeat j ← h(k,i)
3           if T[j] = k
4                   then return j
5          i ← i +1
6   until T[j] = NIL or i = m
7   return NIL
```

- Worst case for inserting a key is $\theta(n)$
- Worst case for searching is $\theta(n)$
- Algorithm assumes that keys are not deleted once they are inserted
- Deleting a key from an open addressing table is difficult, instead we can mark them in the table as removed (introduced a new class of entries, full, empty and removed)

Clustering

- Even with a good hash function, linear probing has its problems:
  - The position of the initial mapping i_0 of key k is called the home position of k.
  - When several insertions map to the same home position, they end up placed contiguously in the table. This collection of keys with the same home position is called a cluster.
  - As clusters grow, the probability that a key will map to the middle of a cluster increases, increasing the rate of the cluster's growth. This tendency of linear probing to place items together is known as primary clustering.
  - As these clusters grow, they merge with other clusters forming even bigger clusters which grow even faster.

Quadratic probing

$h(k,i) = (h'(k) + c_1 i + c_2 i^2) \ mod \ m$ for $i = 0,1,\ldots,m-1$.

- Leads to a secondary clustering (milder form of clustering)
- The clustering effect can be improved by increasing the order to the probing function (cubic). However the hash function becomes more expensive to compute
- But again for two keys k1 and k2, if h(k1,0)= h(k2,0) implies that h(k1,i)= h(k2,i)

Double Hashing

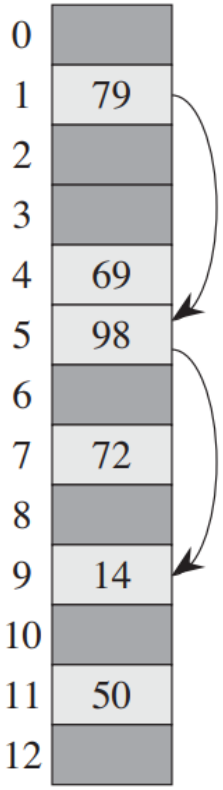
- Recall that in open addressing the sequence of probes follows

$$i_{j+1} = (i_j + c) \bmod m \quad \text{for} \quad j \geq 0$$

- We can solve the problem of primary clustering in linear probing by having the keys which map to the same home position use differing probe sequences. In other words, the different values for c should be used for different keys.
- Double hashing refers to the scheme of using another hash function for c

$$i_{j+1} = (i_j + h_2(k)) \bmod m \quad \text{for} \quad j \geq 0 \quad \text{and} \quad 0 < h_2(k) \leq m - 1$$

Example – Double hashing



$$h(k,i) = (h_1(k) + i\,h_2(k)) \bmod m$$

Figure 11.5 Insertion by double hashing. Here we have a hash table of size 13 with $h_1(k) = k \bmod 13$ and $h_2(k) = 1 + (k \bmod 11)$. Since $14 \equiv 1 \pmod{13}$ and $14 \equiv 3 \pmod{11}$, we insert the key 14 into empty slot 9, after examining slots 1 and 5 and finding them to be occupied.